Predicting Lemmas in Generalization of IC3

Yuheng Su
University of Chinese Academy of Sciences
Institute of Software, Chinese Academy of Sciences
gipsyh.icu@gmail.com

Qiusong Yang*
Institute of Software, Chinese Academy of Sciences
qiusong@iscas.ac.cn

Yiwei Ci
Institute of Software, Chinese Academy of Sciences
yiwei@iscas.ac.cn

ABSTRACT

The IC3 algorithm, also known as PDR, has made a significant impact in the field of safety model checking in recent years due to its high efficiency, scalability, and completeness. The most crucial component of IC3 is inductive generalization, which involves dropping variables one by one and is often the most time-consuming step. In this paper, we propose a novel approach to predict a possible minimal lemma before dropping variables by utilizing the counterexample to propagation (CTP). By leveraging this approach, we can avoid dropping variables if predict successfully. The comprehensive evaluation demonstrates a commendable success rate in lemma prediction and a significant performance improvement achieved by our proposed method.

ACM Reference Format:
Yuheng Su, Qiusong Yang, and Yiwei Ci. 2024. Predicting Lemmas in Generalization of IC3. In *61st ACM/IEEE Design Automation Conference (DAC '24), June 23–27, 2024, San Francisco, CA, USA*. ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3649329.3655970

1 INTRODUCTION

As hardware designs continue to scale and grow increasingly complex, the occurrence of errors in circuits becomes more prevalent, and the cost of such errors is substantial. To ensure the correctness of circuits, formal verification techniques such as symbolic model checking have been widely adopted by the industry. Symbolic model checking enables the proof of properties in hardware circuits or the identification of errors that violate these properties. IC3 [5] (also known as PDR [6]) is a highly influential SAT-based algorithm used in hardware model checking, serving as a primary engine for many state-of-the-art model checkers. In comparison to other symbolic algorithms, IC3 stands out for its completeness compared to BMC and scalability compared to IMC and BDD approaches.

To verify a safety property, IC3 endeavors to compute an inductive invariant. This invariant is derived from a series of frames denoted as $F_0, \cdots, F_k$, where each F_i represents an over-approximation of the set of states reachable in i or fewer steps and does not include any unsafe states. Each frame is represented as a CNF formula, and each clause within the formula is also referred to as a *lemma*. The process terminates when either $F_i = F_{i+1}$ for some i, indicating that an inductive invariant has been reached and the system being verified satisfies the designated safety property, or when a real counterexample is found. The basic algorithm for computing the frame F_{i+1} from the previous F_i first *initializes* the frame F_{i+1} to $\top$, which represents the set of all possible states. It then efficiently *blocks* any unsafe states through inductive generalization and *propagates* the lemmas learned in previous frames to F_{i+1} as far as possible.

Given an unsafe state represented as a boolean cube, the inductive generalization procedure aims to expand it as much as possible to include additional unreachable states. The standard algorithm [5] adopts the "down" strategy by attempting to drop as many literals as possible. If a certain literal is dropped and the negation of the new cube is found to be inductive, this results in a smaller inductive clause. If unsuccessful, the algorithm then attempts the process again with a different literal. This process strengthens the frames and significantly reduces the number of iterations. However, it is also the most time-consuming part and has a significant impact on the performance of IC3. Numerous studies have been undertaken to improve generalizations. According to [7], certain states that are unreachable but can be backtracked from unsafe states, which cannot be further generalized, are identified as counterexamples to generalization (CTG). The negation of a CTG proves to be inductive, leading to a reduction in the depth of the explicit backward search performed by IC3 and enabling stronger inductive generalizations. In [12], the authors first drop literals that have not appeared in any subsumed lemmas of the previous frame in order to increase the probability of producing lemmas that can be effectively propagated to the next frame. In [8], the learned lemmas are labeled as *bad* lemmas if they exclude certain reachable states. Later on, these lemmas are not pushed further and their usage in generalization is minimized.

In this paper, we aim to address the inefficiency of IC3 generalization in the scenario depicted in Figure 1. Let's assume there is only one lemma $\neg c_1$ (where c_1 represents a cube) in the frame F_{i-1} at first. Later, an unsafe state b is reached, and the generalization of b yields the lemma $\neg c_2$. Furthermore, as the lemmas are further propagated to F_i, the lemma $\neg c_1$ succeeds while $\neg c_2$ fails. This failure indicates that the cube c_2 cannot be blocked in F_i. In previous studies[5, 7, 8, 12], the standard practice was to discard the lemma $\neg c_2$ and then rediscover b in an attempt to block and generalize it once again. If we could predict a potential lemma candidate before dropping variable in generalization based on the information learned so far and verify its validity, we might be able to avoid the costly process of dropping variables one by one. However, the

*Qiusong Yang is the corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
DAC '24, June 23–27, 2024, San Francisco, CA, USA
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0601-1/24/06…$15.00
https://doi.org/10.1145/3649329.3655970

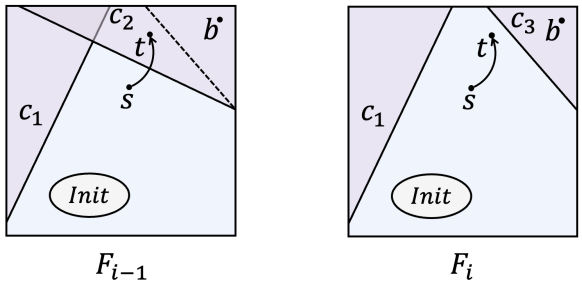

Figure 1: F_{i-1} and F_i frames: the square represents the entire state space, the blue region represents the state space represented by the frame, and each triangle in the purple region represents states blocked by a cube.

challenge lies in developing a prediction mechanism with a high success rate, as an ineffective prediction would result in additional overhead.

The work presented in this paper utilizes counterexample to propagation (CTP) to predict potential lemma candidates and thereby significantly reduce the number of costly generalization operations. As shown in Figure 1, when the lemma $\neg c_2$ cannot be propagated to F_i, it indicates the existence of two states, namely s and t. Here, t is a successor state of s, where $s \models F_{i-1}$ and $t \models c_2$. We will propose a method for predicting candidates for the lemma $\neg c_3$, which blocks the unsafe state b and is inductive relative to F_{i-1}, based on c_2, t, and b. This marks the first implementation of a lemma prediction mechanism in the IC3 algorithm. We conducted a comprehensive performance evaluation of the proposed method, and the results demonstrate that our approach achieves a commendable success rate in predicting lemmas, thereby significantly improving the performance of IC3.

Organization: Section 2 provides some background concepts. In Section 3, the algorithm predicting lemma candidates is presented. The proposed algorithm is comprehensively evaluated in Section 4. Section 5 discusses related works. Section 6 concludes this paper and present directions for future work.

2 PRELIMINARIES

2.1 Basics and Notations

Our context is standard propositional logic. We use notations such as x, y for Boolean variables, and X, Y for sets of Boolean variables. The terms x and $\neg x$ are referred to as *literals*. *Cube* is conjunction of literals, while *clause* is disjunction of literals. The negation of a cube becomes a clause, and vice versa. A Boolean formula in Conjunctive Normal Form (CNF) is a conjunction of clauses. It is often convenient to treat a clause or a cube as a set of literals, and a CNF as a set of clauses. For instance, given a CNF formula F, a clause c, and a literal l, we write $l \in c$ to indicate that l occurs in c, and $c \in F$ to indicate that c occurs in F. A formula F implies another formula G, if every satisfying assignment of F satisfies G, denoted as $F \Rightarrow G$. This relation can also be expressed as $F \models G$.

A Boolean transition system, denoted as S, can be defined as a tuple $\langle X, Y, I, T \rangle$. Here, X and X' represent the sets of state variables in the current state and the next state respectively, while Y represents the set of input variables. The Boolean formula $I(X)$ represents the initial states, and $T(X, Y, X')$ describes the transition relation of the system. State s_2 is a successor of state s_1 iff (s_1, s_2') is an assignment of T ($(s_1, s_2') \models T$). A *path* of length n in S is a finite sequence of states $s_0, s_1, \ldots, s_n$ such that s_0 is an initial state ($s_0 \models I$) and for each i where $0 \leq i < n$, state s_{i+1} is a successor of state s_i ($(s_i, s_{i+1}') \models T$). Checking a safety property of a transition system S is reducible to checking an invariant property [9]. An invariant property $P(X)$ is a Boolean formula over X. A system S satisfies the invariant property P iff all reachable states of S satisfy the property P. Otherwise, there exists a counterexample *path* $s_0, s_1, \ldots, s_k$ such that $s_k \not\models P$.

2.2 Overview of IC3

IC3 is a SAT-based safety model checking algorithm, which only needs to unroll the system at most once [5]. It tries to prove that S satisfies P by finding an inductive invariant $INV(X)$ such that:

- $I(X) \Rightarrow INV(X)$
- $INV(X) \wedge T(X, Y, X') \Rightarrow INV(X')$
- $INV(X) \Rightarrow P(X)$

To achieve this objective, it maintains a monotone CNF sequence $F_0, F_1 \ldots F_k$. Each F_i in the sequence is called a *frame*, which represents an over-approximation of the states reachable in S within i transition steps. Each clause c in F_i is called *lemma* and the index of a frame is called *level*. IC3 maintains the following invariant:

- $F_0 = I$
- for $i > 0$, F_i is a set of clauses
- $F_{i+1} \subseteq F_i$ (F_i, F_{i+1} are sets of clauses)
- $F_i \Rightarrow F_{i+1}$
- $F_i \wedge T \Rightarrow F_{i+1}$
- for $i < k$, $F_i \Rightarrow P$

A lemma c is said to be *inductive relative* to F_i if, starting from the intersection of F_i and c, all states reached in a single transition are located inside c. This condition can be expressed as a SAT query $sat(F_i \wedge c \wedge T \wedge \neg c')$. If this query is satisfied, it indicates that c is not inductive relative to F_i because we can find a counterexample to induction (CTI) that starting from $F_i \wedge c$ and transitioning outside of the lemma c. If lemma c is inductive relative to F_i, it can be also said that cube $\neg c$ is blocked in F_{i+1}.

Algorithm 1 provides an overview of the IC3 algorithm. This algorithm incrementally constructs frames by iteratively performing two phases: the blocking phase and the propagation phase. During the blocking phase, the IC3 algorithm focuses on making $F_k \Rightarrow P$. It iteratively get a cube c such that $c \models \neg P$, and block it recursively. This process involves attempting to block the cube's predecessors if it cannot be blocked directly. It continues until the initial states cannot be blocked, indicating that $\neg P$ can be reached from the initial states in k transitions thus violating the property. In cases where a cube can be confirmed as blocked, IC3 proceeds to enlarge the set of blocked states through a process called generalization. This involves dropping variables and ensuring that the resulting clause remains relative inductive, with the objective of obtaining a minimal inductive clause. Subsequently, IC3 attempts to push the

generalized lemma to the top and incorporates it into the frames. Throughout this phase, the sequence is augmented with additional lemmas, which serve to strengthen the approximation of the reachable state space. The propagation phase tries to push lemmas to the top. If a lemma c in $F_i \setminus F_{i+1}$ is also inductive relative to F_i, then push it into F_{i+1}. During this process, if two consecutive frames become identical ($F_i = F_{i+1}$), then the inductive invariant is found and the safety of this model can be proofed.

3 PREDICTING LEMMAS

3.1 Intuition

The generalization of the IC3 algorithm attempts to iteratively drop variables to obtain a minimal inductive clause. Each variable dropping corresponds to a SAT query, which can incur significant overhead. If there existed a method to speculatively predict a high-quality (minimal) inductive clause and demonstrate its relative inductiveness, it could improve the performance of generalization by avoiding dropping variables one by one.

In Figure 1, when attempting to push the lemma $\neg c_1$ and $\neg c_2$ from F_{i-1} to F_i (where c_1 and c_2 are cubes), c_1 succeeds, but c_2 fails. This failure indicates that the cube c_2 cannot be blocked in F_i. However, it is highly likely that cube c_2 is generalized from state b, where b is a bad state and $b \models c_2$. Since cube c_2 cannot be blocked in F_i, it is possible that b still exists in F_i ($b \models F_i$). IC3 will later find b and attempt to block and generalize it again.

Additionally, in case of a push failure, we encounter two states in counterexample to propagation (CTP): s and t, where t is a successor state of s, $s \models F_{i-1}$, and $t \models c_2$. The existence of CTP prevents c_2 from being blocked in F_i. If we can remove state t from c_2, CTP will no longer be effective to the remaining part, thus the remaining part may potentially be blocked in F_i. In this paper, we attempt to refine c_2 by utilizing state t to derive a new cube c_3, where $t \not\models c_3$, $b \models c_3$ and c_3 can potentially be blocked in F_i, thus a possible lemma in F_i is predicted.

3.2 How to Refine Cubes

How can we refine the cube c_2 using t to obtain a cube c_3 such that $t \not\models c_3$ and $b \models c_3$? To achieve this objective, we introduce the concept of the *diff set* of two cubes:

Definition 3.1 (Diff Set). Let a and b are cubes, $diff(a, b) = \{l \mid l \in a \wedge \neg l \in b\}$.

Diff set $diff(a, b)$ represents the set of literals in a such that their negations are in b. It should be noted that $diff(a, b) \neq diff(b, a)$. The diff set has the following theorems:

Theorem 3.2. *Let a and b are cubes, $a \neq \bot$ and $b \neq \bot$. $a \wedge b = \bot$, iff $diff(a, b) \neq \emptyset$.*

Proof. Let cube $c = a \wedge b$ ($c = a \cup b$). If $c = \bot$, it implies the presence of a literal l in c along with its negation, $\neg l$. Since $a \neq \bot$ and $b \neq \bot$, we can conclude that there exists a literal in a and its negation in b, indicating that $diff(a, b) \neq \emptyset$. Conversely, if $diff(a, b) \neq \emptyset$, we can identify a literal l in $diff(a, b)$, where both l and its negation exist in c, resulting in $a \wedge b = \bot$. □

Theorem 3.3. *Let a, b, c are cubes, if $diff(a, b) \neq \emptyset$ and $c \cap diff(a, b) \neq \emptyset$, then $diff(c, b) \neq \emptyset$.*

Algorithm 1 Overview of IC3

```
1:  function inductive_relative(clause c, level i)
2:      return ¬sat(F_i ∧ c ∧ T ∧ ¬c')
3:
4:  function generalize(cube b, level i)
5:      for each l ∈ b do
6:          cand := b \ {l}                                  ▷ drop variable
7:          if I ⇒ ¬cand and inductive_relative(¬cand, i − 1)
    then
8:              b := cand
9:      return b
10:
11: function block(cube c, level i, level k)
12:     if i = 0 then
13:         return false
14:     while ¬inductive_relative(¬c, i − 1) do
15:         p := get_predecessor(i − 1)
16:         if ¬block(p, i − 1, k) then
17:             return false
18:     mic := ¬generalize(c, i)
19:     while i < k and inductive_relative(mic, i) do
20:         i := i + 1                                       ▷ push lemma
21:     for 1 ≤ j ≤ i do
22:         F_j := F_j ∪ {mic}
23:     return true
24:
25: function propagate(level k)
26:     for 1 ≤ i < k do
27:         for each c ∈ F_i \ F_{i+1} do
28:             if inductive_relative(c, i) then             ▷ push lemma
29:                 F_{i+1} := F_{i+1} ∪ {c}
30:         if F_i = F_{i+1} then
31:             return true
32:     return false
33:
34: procedure ic3(I, T, P)
35:     if sat(I ∧ ¬P) then
36:         return unsafe
37:     F_0 := I, k := 1, F_k := ⊤
38:     while true do
39:         while sat(F_k ∧ ¬P) do                           ▷ blocking phase
40:             c := get_predecessor(k)
41:             if ¬block(c, k, k) then
42:                 return unsafe
43:         k := k + 1, F_k := ⊤                             ▷ propagation phase
44:         if propagate(k) then
45:             return safe
```

Proof. Let l be a literal, and suppose $l \in c \cap diff(a, b)$. It can be derived that $l \in c$. According to Definition 3.1, we know that $l \in a$ and $\neg l \in b$. Since $l \in c$ and $\neg l \in b$, it follows that $l \in diff(c, b)$, leading to the conclusion that $diff(c, b) \neq \emptyset$. □

Additionally, cube has the following theorem:

Theorem 3.4. *Let a and b are cubes, $a \neq \bot$ and $b \neq \bot$. $a \Rightarrow b$ (or $a \models b$) iff $b \subseteq a$.*

Actually in IC3, t and b both are cubes, that they may not only represent one state but a set of states. It is possible for cubes t and b to have intersections. However, we will initially consider the case where there is no intersection, i.e., $t \wedge b = \bot$. According to Theorem 3.2, we can derive the following equation:

$$diff(b, t) \neq \emptyset \tag{1}$$

Our objective is to find a cube c_3 that satisfies $c_3 \wedge t = \bot$, $b \models c_3$, and $c_3 \models c_2$. Referring to Theorem 3.2 and Theorem 3.4, we can achieve our objective if c_3 satisfies the following equations:

$$diff(c_3, t) \neq \emptyset \tag{2}$$

$$c_3 \subseteq b \tag{3}$$

$$c_2 \subseteq c_3 \tag{4}$$

Building upon Theorem 3.3 and Equation 1, we can establish the truth of Equation 2 by ensuring the satisfaction of the following equation:

$$c_3 \cap diff(b, t) \neq \emptyset \tag{5}$$

By constructing c_3 according to the following equation, we can satisfy Equation 5:

$$c_3 = c_2 \cup \{l\}\ (l \in diff(b, t)) \tag{6}$$

The method we use for predicting lemmas is represented by Equation 6. This equation demonstrates the process of attempting to select an element from the diff set and incorporate it into $c2$, resulting in the creation of a new cube $c3$ that is larger than $c2$ by one element. If Equation 6 is satisfied, it is evident that Equation 4 is true. Additionally, based on Definition 3.1, Equation 3 is also satisfied. As a result, we have successfully obtained the desired cube.

3.3 The Algorithm

Algorithm 2 introduces our proposed approach for lemma prediction by leveraging the CTP in IC3. The modifications to the original IC3 algorithm are denoted by the blue-colored code. In line 36 and line 47, the algorithm attempts to push lemmas. If push fails, it retrieves the corresponding state t from the SAT solver and stores it in a hash table. The hash table uses a tuple consisting of the failed lemma and the current level as the key. Additionally, at regular intervals before each propagation step, the hash table is cleared and reconstructed.

In the generalization, the algorithm first attempts to predict a possible minimal lemma and checks its validity, as demonstrated in lines 10 to 27. Initially, it identifies all the parent lemmas of clause $\neg b$ in level i. A *parent lemma* of clause c at level i is the lemma p only in the frame of level $i-1$ (not in the current level) that $p \Rightarrow c$. For example, it can be observed that $\neg c_2$ is the parent lemma of clause $\neg b$ at level i in Figure 1. We make an attempt to predict a minimal lemma in F_i based on each identified parent lemma:

(1) If the parent lemma p does not have any failed push attempts at level $i-1$, we cannot find CTP, and thus cannot predict a lemma and proceed with the next parent lemma as shown in lines 12 - 13.

Algorithm 2 Predicting Lemmas

```
1:  function parent_lemmas(clause c, level i)
2:      parents := ∅
3:      if i ≠ 0 then
4:          for each p ∈ F_i \ F_{i+1} do
5:              if p ⊆ c then
6:                  parents := parents ∪ {p}
7:      return parents
8:
9:  function generalize(cube b, level i)
10:     parents := parent_lemmas(¬b, i − 1)
11:     for each p ∈ parents do
12:         if ¬failure_push.has(p, i − 1) then        ▷ find state t
13:             continue
14:         t := failure_push[(p, i − 1)]
15:         ds := diff(b, t)                           ▷ get diff set
16:         if ds = ∅ then                             ▷ push parent lemma
17:             if inductive_relative(p, i − 1) then
18:                 return p
19:             else
20:                 failure_push[(p, i − 1)] := get_model(i − 1)
21:         else
22:             for each d ∈ ds do
23:                 cand := p ∪ {¬d}
24:                 if inductive_relative(cand, i − 1) then
25:                     return ¬cand
26:                 else
27:                     ds := ds ∩ diff(b, get_model(i − 1))
28:     for each l ∈ b do
29:         cand := b \ {l}                            ▷ drop variable
30:         if I ⇒ ¬cand and inductive_relative(¬cand, i − 1)
    then
31:             b := cand
32:     return b
33:
34: function block(cube c, level i, level k)
35:     ...
36:     while i < k and inductive_relative(mic, i) do
37:         i := i + 1                                 ▷ push lemma
38:     failure_push[(mic, i)] := get_model(i)         ▷ record state t
39:     for 1 ≤ j ≤ i do
40:         F_j := F_j ∪ {mic}
41:     return true
42:
43: function propagate(level k)
44:     failure_push.clear()                           ▷ reconstruct hash table
45:     for 1 ≤ i < k do
46:         for each c ∈ F_i \ F_{i+1} do
47:             if inductive_relative(c, i) then       ▷ push lemma
48:                 F_{i+1} := F_{i+1} ∪ {c}
49:             else                                   ▷ record state t
50:                 failure_push[(c, i)] := get_model(i)
51:         if F_i = F_{i+1} then
52:             return true
53:     return false
```

(2) Upon successfully retrieving state t from the hash table, we proceed to calculate the diff set ds of the cube b and cube t. In lines 16 - 20, if the diff set is empty, it indicates an intersection between the cubes b and t according to Theorem 3.2, meaning that some states in cube t are already blocked due to the blocking of b in F_i. In this case, we consider pushing the parent lemma p to F_i as a predicted lemma. If successful, the predicted lemma is deemed valid, we take it as the final result. Otherwise, we store the state t associated with this failure in the hash table.

(3) If the diff set is not empty, we attempt to predict a possible lemma using Equation 6 by iteratively selecting a literal from the diff set as shown in lines 22 - 27. If successful, since it is only one variable longer than the parent lemma, we consider it to be a high-quality lemma. Therefore, we no longer generalize it by dropping variables and directly treat it as the final result of generalization. If unsuccessful, it is highly likely that the counterexample is also an another CTP of pushing p to F_i, thus we recalculate the diff set and eliminate certain infeasible candidates from it.

4 EVALUATION

4.1 Experimental Setup

IC3ref [3] is an IC3 implementation provided by the inventor of this algorithm, which is competitive and commonly used as the baseline [10–12]. We also implemented IC3 algorithm in Rust, a modern programming language designed to offer both performance and safety, denoted as *RIC3*, which is competitive to IC3ref. We integrated our proposed predicting lemmas optimization into both IC3ref and RIC3, denoted as *IC3ref-pl* and *RIC3-pl* respectively. Since our method is related to [12], which is also implemented in IC3ref, available at [2], we also conducted an evaluation of it, denoted as *IC3ref-CAV23*. We also consider the PDR implementation in the ABC [1], which is a competitive model checker.

We conducted all experiments using the complete benchmark set of HWMCC'15 (excluding restricted-access cases) and HWMCC'17, totaling 730 cases, under consistent resource constraints (8 GB, 1000s). The experiments were performed on an AMD EPYC 7532 machine with 32 cores running at 2.4 GHz. To ensure reproducibility, we have provided our experiment implementations [4].

4.2 Experimental Results

The summary of results, comparisons among different configurations, and scatter plots of implementations with and without lemma prediction are presented in Table 1, Figure 2, and Figure 3, respectively. It can be observed that by incorporating our proposed optimization, different implementations solve more cases compared to the original implementation within various time limits and a greater number of cases have been solved faster. It can also be observed that our proposed method solves more cases than IC3ref-CAV23 and ABC-PDR. This demonstrates the effectiveness of our proposed method in enhancing the performance of IC3.

4.3 Analysis

To better demonstrate the effectiveness of our proposed method, we introduced several success rates: the lemma prediction success rate ($SR_{lp} = N_{sp}/N_p$), the success rate of finding failed pushed parent lemmas ($SR_{fp} = N_{fp}/N_g$), and the success rate of avoiding dropping variables ($SR_{adv} = N_{sp}/N_g$). Here, N_{sp} represents the number of successful lemma predictions, N_p represents the total number of lemma predictions (the number of SAT queries in prediction), N_{fp} represents the number of successfully finding failed pushed

Table 1: Summary of Results

Configuration	Solved	Safe	Unsafe
RIC3	365	264	101
RIC3-pl	375	273	102
IC3ref	371	263	108
IC3ref-pl	379	268	111
IC3ref-CAV23	375	269	106
ABC-PDR	373	267	106

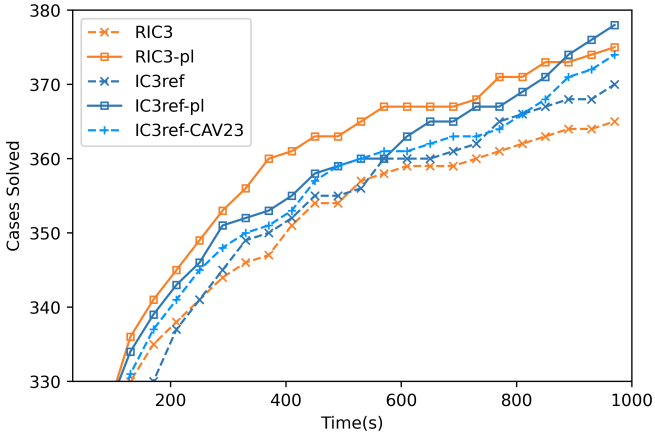

Figure 2: Comparisons among the different configurations.

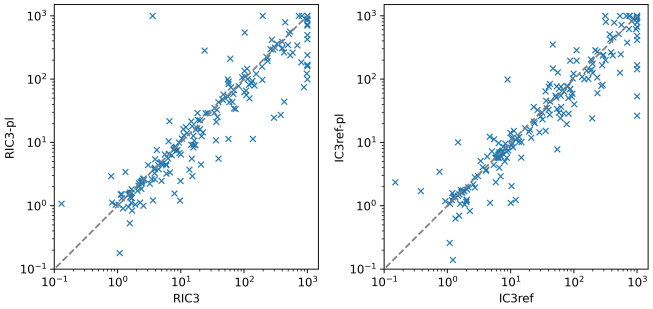

Figure 3: Scatters of RIC3 and IC3ref with and without the proposed optimization. Points below the diagonal indicate better performance with the proposed optimization active.

Table 2: Average Success Rates

Configuration	Avg SR_{lp}	Avg SR_{fp}	Avg SR_{adv}
RIC3-pl	38.61%	40.67%	24.03%
IC3ref-pl	31.5%	37.81%	19.46%

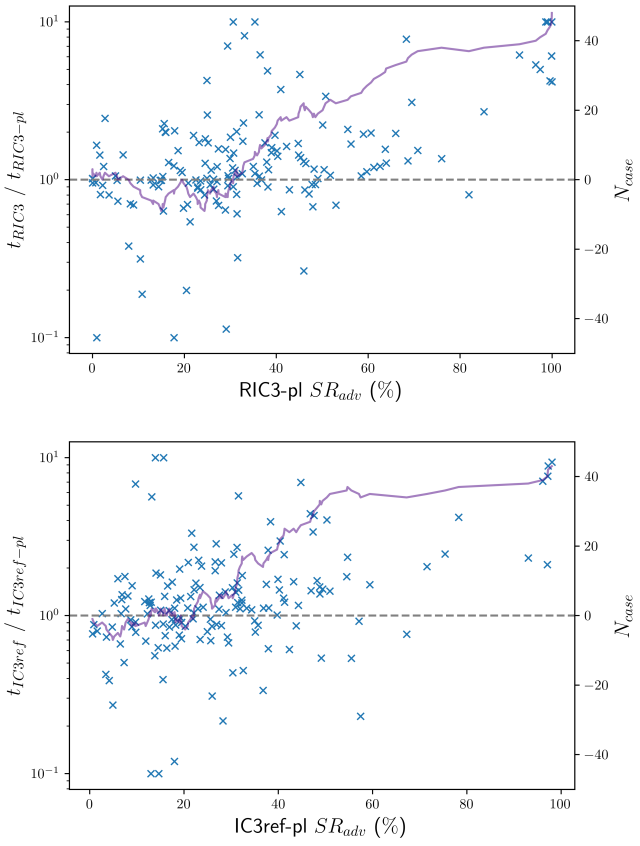

Figure 4: Comparing the ratio of runtime of different implementations without and with our proposed optimization (left y-axis) with the success rate of avoiding dropped variables SR_{adv} during generalization (x-axis). The right y-axis represents the cumulative number of cases with improved performance as the prediction accuracy increases. Cases with runtime both timeout or both less than 1s are ignored.

parent lemmas, and N_g represents the total number of generalizations. Table 2 presents the three kind of average success rates of different implementations with our proposed optimization. It can be observed that our method can achieve a considerable success rate if the failed pushed lemma is found. Figure 4 depicts the correlation between SR_{adv} and the checking time. It can be observed that higher prediction success rates result in a better performance in the proportion of cases. This observation supports the conjecture that higher accuracy in predictions can yield better performance.

5 RELATED WORK

After the introduction of IC3 [5], numerous works have been proposed to improve the efficiency. [10] interleaves a forward and a backward execution of IC3 and strengthens one frame sequence by leveraging the other. [11] introduces under-approximation to improve the performance of bug-finding.

When a clause fails to be relatively inductive, a common approach is to identify the counterexample of this failure and attempting to render it ineffective. [7] attempts to block counterexample to generalization (CTG) to make the variable dropped clause inductive. [8] aggressively pushes lemmas to the top by adding may-proof-obligation (blocking counterexamples of push failure). The key distinction between our proposed method and these approaches lies in the fact that while they all focus on blocking the predecessor state (state s in Figure 1) of a counterexample, we utilize the successor state (state t) to predict a possible lemma.

Another related work is [12], which also attempts to find the parent lemma and doesn't dropping literals which existed in the parent lemma during generalization. Our approach differs significantly as we attempt to avoid dropping variables by refining failed pushed parent lemma as a prediction.

6 CONCLUSION AND FUTURE WORK

In this paper, we proposed a novel method for predicting minimal lemmas before dropping variables by utilizing the counterexample to propagation (CTP). By making successful predictions, we can effectively circumvent the need to drop variables during generalization, which is a significant overhead in the IC3 algorithm. The experimental results demonstrate a commendable success rate in lemma prediction, consequently leading to a significant performance improvement. In the future, we plan to improve the prediction rate of our proposed method in order to avoiding dropping variables as much as possible.

ACKNOWLEDGMENTS

This work was supported by the Basic Research Projects from the Institute of Software, Chinese Academy of Sciences (Grant No. ISCAS-JCZD-202307) and the National Natural Science Foundation of China (Grant No. 62372438).

REFERENCES

[1] 2023. ABC. https://github.com/berkeley-abc/abc.
[2] 2023. CAV23 Artifact. https://github.com/youyusama/i-Good_Lemmas_MC.
[3] 2023. IC3ref. https://github.com/arbrad/IC3ref.
[4] 2023. Reproductive Artifact. https://github.com/gipsyh/DAC24.
[5] Aaron R. Bradley. 2011. SAT-Based Model Checking without Unrolling. In *Proceedings of the 12th International Conference on Verification, Model Checking, and Abstract Interpretation* (Austin, TX, USA) *(VMCAI'11)*. Springer-Verlag, Berlin, Heidelberg, 70–87.
[6] Niklas Een, Alan Mishchenko, and Robert Brayton. 2011. Efficient implementation of property directed reachability. In *2011 Formal Methods in Computer-Aided Design (FMCAD)*. 125–134.
[7] Zyad Hassan, Aaron R. Bradley, and Fabio Somenzi. 2013. Better generalization in IC3. In *2013 Formal Methods in Computer-Aided Design*. 157–164. https://doi.org/10.1109/FMCAD.2013.6679405
[8] Alexander Ivrii and Arie Gurfinkel. 2015. Pushing to the top. In *2015 Formal Methods in Computer-Aided Design (FMCAD)*. 65–72.
[9] Zohar Manna and Amir Pnueli. 1995. *Temporal Verification of Reactive Systems: Safety*. Springer-Verlag, Berlin, Heidelberg.
[10] Tobias Seufert and Christoph Scholl. 2019. fbPDR: In-depth combination of forward and backward analysis in Property Directed Reachability. In *2019 Design, Automation Test in Europe Conference Exhibition (DATE)*. 456–461.
[11] Tobias Seufert, Christoph Scholl, Arun Chandrasekharan, Sven Reimer, and Tobias Welp. 2022. Making PROGRESS in Property Directed Reachability. In *Verification, Model Checking, and Abstract Interpretation*, Bernd Finkbeiner and Thomas Wies (Eds.). Springer International Publishing, Cham, 355–377.
[12] Yechuan Xia, Anna Becchi, Alessandro Cimatti, Alberto Griggio, Jianwen Li, and Geguang Pu. 2023. Searching for i-Good Lemmas to Accelerate Safety Model Checking. In *Computer Aided Verification*. Springer Nature Switzerland, Cham, 288–308.